

Projet de Recherche Opérationnelle

Applications classiques, logistiques et réelles

Module : Recherche Opérationnelle - Licence

Année universitaire : 2025-2026

Étudiant :

Objectif général

Ce rapport présente plusieurs problèmes de recherche opérationnelle. L'objectif est de modéliser chaque problème, proposer une méthode de résolution, exécuter un exemple numérique avec Python et interpréter les résultats obtenus.

Sujets traités

N	Sujet	Catégorie	Fichier Python
1	Optimisation du transport	Sujets classiques	transport.py
2	Problème du voyageur (TSP)	Logistique	tsp.py
3	Optimisation combinatoire - Greedy	Méthodes numériques	heuristique_tsp.py
4	Files d'attente en centre hospitalier	Problèmes réels RIM	hospital_queue.py
5	Routage des camions d'eau	Problèmes réels RIM	water_truck_routing.py
6	Transport de poissons	Problèmes réels RIM	poisson_transport.py

1. Optimisation du transport

Description

Le problème de transport consiste à distribuer des marchandises depuis plusieurs usines vers plusieurs entrepôts en minimisant le coût total. Il est très utilisé dans la logistique, la distribution et la gestion des stocks.

Modélisation et méthode

Variables : x_{ij} représente la quantité envoyée de l'usine i vers l'entrepôt j .

Objectif : $\text{Min } Z = \sum_{i,j} c_{ij} x_{ij}$.

Contraintes : respect des capacités des usines, satisfaction des demandes des entrepôts et positivité des variables.

Tableau des coûts unitaires

	E1	E2	E3
U1	2	4	5
U2	3	1	7

Résultat Python

transport.py - Résultat

Statut : Optimal

$x_{11} = 50.0$ $x_{12} = 0.0$ $x_{13} = 50.0$

$x_{21} = 30.0$ $x_{22} = 120.0$ $x_{23} = 0.0$

Coût total minimal = 560.0

Interprétation

La solution indique combien envoyer de chaque usine vers chaque entrepôt. Le coût minimal obtenu est 560. Cela montre que la modélisation linéaire permet de réduire les coûts de transport.

2. Problème du voyageur (TSP)

Description

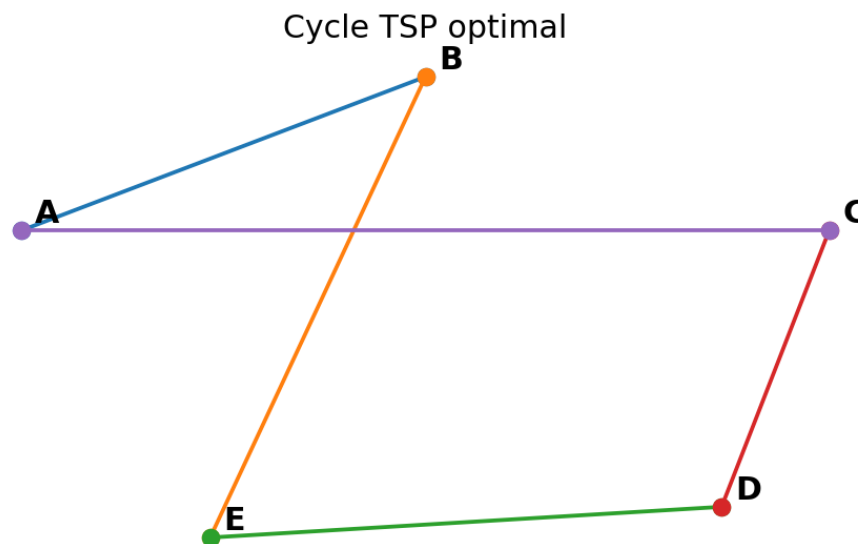
Le TSP consiste à trouver un cycle de longueur minimale visitant toutes les villes une seule fois avant de revenir à la ville de départ. Ce problème apparaît dans les tournées de livraison et la planification de déplacements.

Modélisation et méthode

Variables : $x_{ij} = 1$ si le trajet de i vers j est choisi, 0 sinon.

Objectif : minimiser la distance totale parcourue.

Contraintes : chaque ville doit être visitée exactement une fois et il faut éviter les sous-cycles.



Résultat Python

tsp.py - Résultat exact

Meilleur tour : A -> B -> E -> D -> C -> A
Distance minimale : 84

Interprétation

La méthode exacte teste toutes les permutations dans ce petit exemple. La meilleure distance est 84. Pour des instances plus grandes, il faut utiliser des méthodes plus efficaces comme branch-and-bound ou des heuristiques.

3. Optimisation combinatoire et heuristique greedy

Description

L'optimisation combinatoire traite des problèmes avec un très grand nombre de solutions possibles. Les heuristiques permettent d'obtenir rapidement une bonne solution sans garantir toujours l'optimum.

Modélisation et méthode

Méthode greedy : à chaque étape, on choisit la ville la plus proche non encore visitée. Cette stratégie est rapide et simple à expliquer, mais elle peut donner une solution différente de la solution optimale.

Résultat Python

heuristique_tsp.py - Résultat

Tour trouvé par heuristique greedy :
A -> B -> E -> D -> C -> A
Distance totale : 84

Interprétation

Dans l'exemple, l'heuristique trouve un cycle de distance 84. Ici, elle atteint la même distance que la méthode exacte, ce qui montre son efficacité sur cette instance.

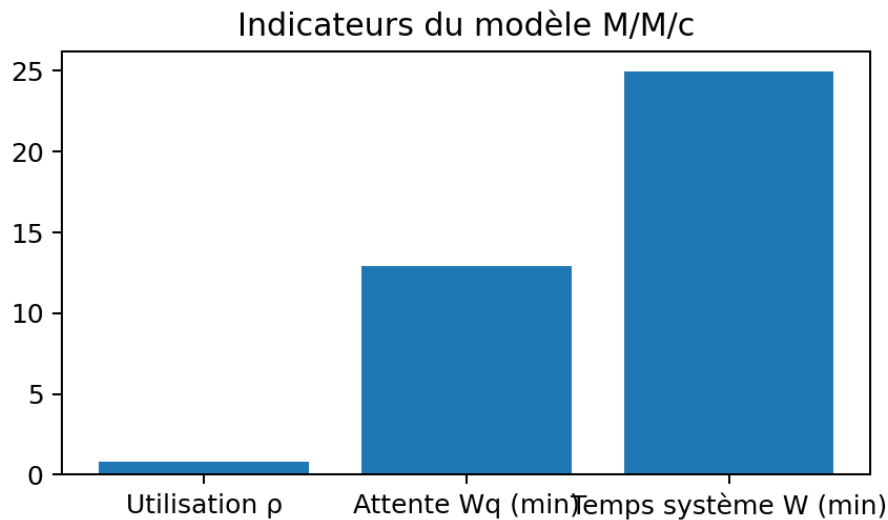
4. Files d'attente en centre hospitalier

Description

Ce sujet réel étudie l'attente des patients dans un centre hospitalier. L'objectif est d'évaluer la charge du système et de réduire le temps d'attente en choisissant un nombre adapté de serveurs.

Modélisation et méthode

Modèle M/M/c : λ est le taux d'arrivée, μ le taux de service et c le nombre de serveurs. L'indicateur principal est $\rho = \lambda / (c \mu)$. Si ρ est proche de 1, le système est fortement chargé.



Résultat Python

hospital_queue.py - Résultat

```
Modèle M/M/c - Centre hospitalier
Taux arrivée  $\lambda = 12$  patients/heure
Taux service  $\mu = 5$  patients/heure, serveurs  $c = 3$ 
Utilisation  $\rho = 0.800$ 
 $L_q = 2.589$  patients
 $W_q = 12.94$  minutes,  $W = 24.94$  minutes
```

Interprétation

Avec $\lambda = 12$ patients/heure, $\mu = 5$ patients/heure et $c = 3$, l'utilisation est 0,8. Le système est stable, mais l'attente moyenne reste importante. Une amélioration possible est d'ajouter un guichet ou d'organiser mieux les consultations.

5. Routage des camions d'eau

Description

Ce problème réel consiste à planifier les tournées des camions d'eau dans des quartiers périphériques en minimisant la distance totale tout en respectant la capacité du camion.

Modélisation et méthode

On utilise une heuristique du plus proche voisin avec contrainte de capacité. Le camion part du dépôt, visite les points possibles sans dépasser 1000 L, puis revient au dépôt si nécessaire.

Résultat Python

water_truck_routing.py - Résultat

Routage des camions d'eau
Route proposée : Depot -> P1 -> P3 -> P5 -> P6 -> Depot -> P2 -> P4 -> Depot
Distance totale : 46 km
Capacité camion : 1000 L

Interprétation

La route proposée respecte la capacité du camion et réduit les déplacements inutiles. Cette méthode peut aider à améliorer la distribution quotidienne d'eau dans les quartiers lointains.

6. Transport de poissons Nouadhibou - Nouakchott

Description

Le transport de poissons frais nécessite une organisation efficace à cause des contraintes de coût, de capacité et de conservation. Les camions frigorifiques sont importants pour préserver la qualité du poisson.

Modélisation et méthode

Variables : nombre de camions normaux et nombre de camions frigorifiques.

Objectif : minimiser le coût total.

Contraintes : satisfaire la demande de 18 tonnes, respecter les limites de camions et privilégier une solution faisable.

Résultat Python

poisson_transport.py - Résultat

Transport de poissons Nouadhibou - Nouakchott
Statut : Optimal
Nombre de camions normaux : 0
Nombre de camions frigorifiques : 6
Coût total minimal : 54000 MRU

Interprétation

Le résultat propose 6 camions frigorifiques pour transporter 18 tonnes avec un coût minimal de 54000 MRU. Cette solution est cohérente avec l'objectif de préserver le produit pendant le transport.

Conclusion générale

La recherche opérationnelle permet de transformer des problèmes réels en modèles mathématiques exploitables. Dans ce projet, les modèles de transport, de routage, de files d'attente et d'heuristiques montrent comment réduire les coûts, les distances et les temps d'attente. L'utilisation de Python permet aussi de vérifier les résultats et de rendre le travail plus expérimental.

Annexes - Exécution

Les fichiers Python peuvent être exécutés avec les commandes suivantes :

```
python transport.py
python tsp.py
python heuristique_tsp.py
python hospital_queue.py
python water_truck_routing.py
python poisson_transport.py
```